

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN KỸ THUẬT PHẦN MỀM



KIẾN TRÚC PHẦN MỀM

Order Service - Clean Architecture Documentation
(Phiên bản 1.0)

Ngành: Kỹ thuật phần mềm

CBGD: Từ Lãng Phiêu

TP. Hồ Chí Minh, tháng 2 năm 2025



MỤC LỤC

1	Tổng quan hệ thống	2
1.1	Mục tiêu dự án	2
1.2	Technology Stack	2
1.3	Kiến trúc tổng thể	3
2	Kiến trúc Clean Architecture	4
2.1	Dependency Inversion Principle (DIP)	4
2.2	Ports & Adapters Pattern	5
2.2.1	Input Ports (Primary/Driving)	5
2.2.2	Output Ports (Secondary/Driven)	6
3	Luồng xử lý chi tiết	7
3.1	Create Order - Sequence Diagram	7
3.2	Step-by-Step Explanation với Code	7
3.2.1	BƯỚC 1: Customer gửi HTTP Request	7
3.2.2	BƯỚC 2: OrderController nhận request	8
4	Cấu trúc module	11
4.1	Multi-module Maven Project	11
4.2	Dependency Matrix	11
5	Implementation chi tiết	14
5.1	Repository Implementation	14
5.2	Domain Entity Implementation	15
6	Database Schema	18
6.1	PostgreSQL Schema Design	18
7	SAGA Pattern	20
7.1	Distributed Transaction Problem	20
8	Testing Strategy	21
8.1	Testing Pyramid	21
8.2	Unit Test Example	21
9	Deployment	23
9.1	Docker Configuration	23
9.2	Docker Compose Setup	24



10 Câu hỏi mở rộng cho sinh viên	27
10.1 Phần 1: Hiểu rõ Clean Architecture	27
10.2 Phần 2: Triển khai thêm tính năng	27
10.3 Phần 3: Cải thiện hệ thống	27
10.4 Phần 4: Testing	28
10.5 Phần 5: Security	28
11 Next Steps	29
11.1 Next Steps cho Sinh viên	29
THUẬT NGỮ VÀ CHỮ VIẾT TẮT	30
TÀI LIỆU THAM KHẢO	31



1 Tổng quan hệ thống

1.1 Mục tiêu dự án

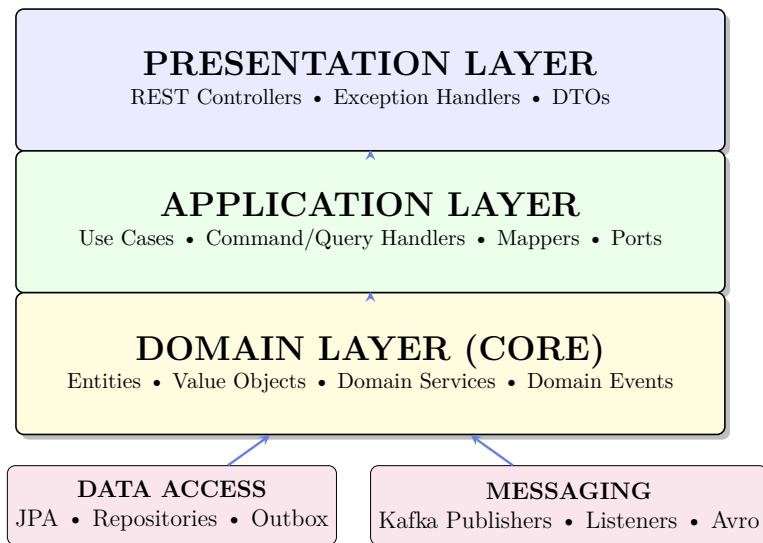
Order Service là một microservice trong hệ thống Food Ordering System, được thiết kế theo **Clean Architecture** (Hexagonal Architecture/Ports & Adapters) với các mục tiêu:

- **Quản lý đơn hàng:** Tạo, theo dõi, hủy đơn hàng từ khách hàng
- **Xử lý workflow:** Orchestrate quy trình tạo đơn → thanh toán → phê duyệt
- **Event-driven communication:** Tích hợp với Payment Service và Restaurant Service qua Kafka
- **Distributed transactions:** Đảm bảo consistency với SAGA Pattern
- **Maintainability:** Code dễ hiểu, dễ test, dễ thay đổi

1.2 Technology Stack

Category	Technology	Version	Purpose
Language	Java	17	Programming language
Framework	Spring Boot	3.2.0	Application framework
ORM	Spring Data JPA	3.2.0	Database access
Database	PostgreSQL	14	Data persistence
Messaging	Apache Kafka	3.5.0	Event streaming
Schema	Avro	1.11.3	Message serialization
Build Tool	Maven	3.9.0	Multi-module build
Testing	JUnit 5 + Mockito	5.10.0	Unit & Integration tests
Containers	Docker + Docker Compose	Latest	Deployment

Bảng 1: Technology Stack



Dependency Rule: Outer layers depend on inner layers — **never the reverse.**

Hình 1: Clean Architecture Overview

1.3 Kiến trúc tổng thể

Nguyên tắc quan trọng nhất

DEPENDENCY RULE: Tất cả dependencies chỉ trở vào trong (towards Domain Layer)

- Domain Layer không phụ thuộc vào bất kỳ layer nào khác
- Application Layer phụ thuộc vào Domain Layer
- Infrastructure Layer (DataAccess, Messaging) phụ thuộc vào Domain và Application
- Container phụ thuộc vào tất cả các layers



2 Kiến trúc Clean Architecture

2.1 Dependency Inversion Principle (DIP)

Dependency Inversion là nền tảng của Clean Architecture. Thay vì:

Cách truyền thống (BAD)

```
// High-level module depends on low-level module
public class OrderService {
    // Direct dependency on concrete class
    private MySQLOrderRepository repository = new MySQLOrderRepository();

    public void createOrder(Order order) {
        repository.save(order); // Coupled to MySQL
    }
}
```

Vấn đề: Muốn đổi database phải sửa OrderService

Clean Architecture (GOOD)

```
// Step 1: Domain defines interface (Output Port)
public interface OrderRepository {
    Order save(Order order);
    Optional<Order> findById(OrderId id);
}

// Step 2: Application uses interface
public class OrderService {
    private final OrderRepository repository; // Interface, not concrete

    // Constructor injection
    public OrderService(OrderRepository repository) {
        this.repository = repository;
    }

    public void createOrder(Order order) {
        repository.save(order); // Not aware of MySQL or MongoDB
    }
}

// Step 3: Infrastructure implement interface
@Component
public class MySQLOrderRepository implements OrderRepository {
    @Override
    public Order save(Order order) {
        // MySQL specific implementation
    }
}
```

Lợi ích: Thay đổi database chỉ cần tạo implementation mới, không động đến OrderService

2.2 Ports & Adapters Pattern

2.2.1 Input Ports (Primary/Driving)

Định nghĩa **cách bên ngoài điều khiển** application:

```
/**
 * INPUT PORT - Interface for Use Cases
```

```
*  
* Location: order-application-service/ports/input/service  
*  
* Notes:  
* - This interface is the application's entry point  
* - REST Controller, GraphQL Resolver, gRPC Service can call it  
* - Domain is unaware of HTTP, REST, GraphQL...  
*/  
public interface OrderApplicationService {  
    // Use Case: Create order  
    CreateOrderResponse createOrder(CreateOrderCommand command);  
  
    // Use Case: Track order  
    TrackOrderResponse trackOrder(TrackOrderQuery query);  
  
    // Use Case: Cancel order  
    CancelOrderResponse cancelOrder(CancelOrderCommand command);  
}
```

Listing 1: Input Port - Interface cho Use Cases

2.2.2 Output Ports (Secondary/Driven)

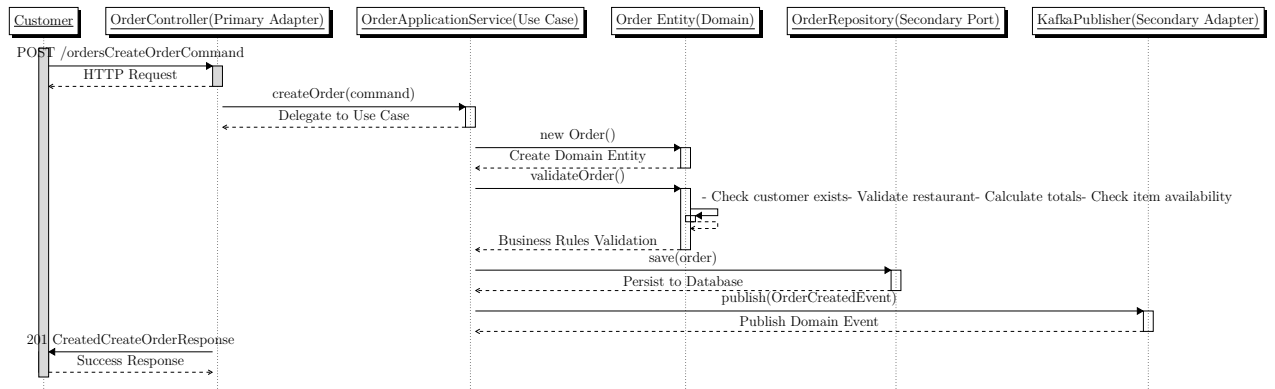
Định nghĩa **dependencies** mà application cần:

```
/**  
* OUTPUT PORT - Interface for dependencies  
*  
* Location: order-application-service/ports/output/repository  
*/  
public interface OrderRepository {  
    // Persist order - agnostic of MySQL, MongoDB, or InMemory  
    Order save(Order order);  
  
    // Find by ID  
    Optional<Order> findById(OrderId orderId);  
  
    // Find by tracking ID  
    Optional<Order> findByTrackingId(TrackingId trackingId);  
}
```

Listing 2: Output Port - Interface cho dependencies

3 Luồng xử lý chi tiết

3.1 Create Order - Sequence Diagram



Hình 2: Create Order - Detailed Sequence Diagram

Sequence Flow Explanation

Clean Architecture Flow:

1. **HTTP Request:** Customer gửi POST request với order data
2. **Primary Adapter:** OrderController nhận request, convert thành Command
3. **Use Case:** OrderApplicationService xử lý business workflow
4. **Domain Entity:** Tạo Order entity và validate business rules
5. **Secondary Port:** Lưu order vào database thông qua Repository interface
6. **Event Publishing:** Publish OrderCreatedEvent để notify các services khác
7. **Response:** Trả về success response cho customer

Key Benefits: Dependency Rule được tuân thủ - Domain không biết về HTTP, Database hay Kafka!

3.2 Step-by-Step Explanation với Code

3.2.1 BƯỚC 1: Customer gửi HTTP Request

```
POST http://localhost:8181/orders
Content-Type: application/json
```

```
{
  "customerId": "d215b5f8-0249-4dc5-89a3-51fd148cfb41",
  "restaurantId": "d215b5f8-0249-4dc5-89a3-51fd148cfb45",
  "price": 200.00,
  "items": [
    {
      "productId": "d215b5f8-0249-4dc5-89a3-51fd148cfb48",
      "quantity": 1,
      "price": 50.00,
      "subTotal": 50.00
    }
  ],
  "address": {
    "street": "123 Main Street",
    "postalCode": "1000AB",
    "city": "Amsterdam"
  }
}
```

Listing 3: HTTP Request Example

3.2.2 BƯỚC 2: OrderController nhận request

```
@RestController
@RequestMapping(value = "/orders", produces = "application/vnd.api.v1+json")
@Slf4j
public class OrderController {

    // Inject Use Case interface (Input Port)
    private final OrderApplicationService orderApplicationService;

    /**
     * Constructor Injection
     *
     * Benefits:
     * - Testable: can inject mocks in unit tests
     * - Immutable: final fields cannot change after construction
     * - Clear dependencies: explicit requirements
     */
}
```

```
public OrderController(OrderApplicationService orderApplicationService) {
    this.orderApplicationService = orderApplicationService;
}

/**
 * PRIMARY ADAPTER
 *
 * Responsibilities:
 * 1. Receive HTTP request
 * 2. Validate input (with @Valid)
 * 3. Convert request -> Command object
 * 4. Delegate to Use Case
 * 5. Convert result -> HTTP response
 */
@PostMapping
public ResponseEntity<CreateOrderResponse> createOrder(
    @Valid @RequestBody CreateOrderCommand createOrderCommand) {

    // Log request
    log.info("Creating order for customer: {} at restaurant: {}",
        createOrderCommand.getCustomerId(),
        createOrderCommand.getRestaurantId());

    // Delegate to use case (Input Port)
    CreateOrderResponse response =
        orderApplicationService.createOrder(createOrderCommand);

    // Log success
    log.info("Order created with tracking id: {}",
        response.getOrderTrackingId());

    // Return HTTP 201 Created
    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(response);
}
```

Listing 4: OrderController - Primary Adapter



Controller Best Practices

- Thin controllers - chỉ handle HTTP concerns
- Không chứa business logic
- Use @Valid để validate input
- Log quan trọng events
- Return appropriate HTTP status codes



4 Cấu trúc module

4.1 Multi-module Maven Project

Order Service is organized as a **multi-module Maven** project to enforce the dependency rule:

4.2 Dependency Matrix

Module	Depends On	Purpose	Framework
order-domain-core	None	Pure business logic	Pure Java
order-application-service	order-domain-core	Use cases & ports	Pure Java
order-dataaccess	order-domain-core order-application-service	Database access	Spring Data JPA
order-messaging	order-domain-core order-application-service	Event messaging	Spring Kafka
order-container	All modules	Web layer & config	Spring Boot

Bảng 2: Module Dependencies



```
order-service/
├── pom.xml # Parent POM
├── order-domain/ # DOMAIN LAYER
│   ├── order-domain-core/
│   │   ├── entity/ # Entities & Aggregates
│   │   ├── valueobject/ # Value Objects
│   │   ├── event/ # Domain Events
│   │   ├── exception/ # Domain Exceptions
│   │   └── service/ # Domain Services
│   └── order-application-service/ # APPLICATION LAYER
│       ├── dto/ # Commands, Queries, Responses
│       ├── ports/ # Input & Output Ports
│       ├── mapper/ # Data Mappers
│       └── handler/ # Use Case Handlers
├── order-dataaccess/ # INFRASTRUCTURE - Data
│   ├── adapter/ # Repository Implementations
│   ├── entity/ # JPA Entities
│   ├── repository/ # JPA Repositories
│   └── mapper/ # JPA Mappers
├── order-messaging/ # INFRASTRUCTURE - Events
│   ├── publisher/ # Kafka Publishers
│   ├── listener/ # Kafka Listeners
│   ├── mapper/ # Message Mappers
│   └── config/ # Kafka Configuration
├── order-container/ # MAIN APPLICATION
│   └── rest/ # REST Controllers
└── config/ # Spring Configuration
    └── OrderServiceApplication.java # Main Class
```



Dependency Enforcement

Maven enforcer plugin đảm bảo không vi phạm dependency rule:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-enforcer-plugin</artifactId>
  <executions>
    <execution>
      <id>enforce-banned-dependencies</id>
      <goals>
        <goal>enforce</goal>
      </goals>
      <configuration>
        <rules>
          <bannedDependencies>
            <excludes>
              <exclude>*:spring-*</exclude> <!-- Domain must not depend on
                Spring -->
            </excludes>
          </bannedDependencies>
        </rules>
      </configuration>
    </execution>
  </executions>
</plugin>
```

5 Implementation chi tiết

5.1 Repository Implementation

```
@Component
public class OrderRepositoryImpl implements OrderRepository {

    // JPA Repository - generated by Spring Data
    private final OrderJpaRepository orderJpaRepository;
    // Mapper: Domain Entity <-> JPA Entity
    private final OrderDataAccessMapper orderDataAccessMapper;

    /**
     * ADAPTER PATTERN
     *
     * Transformations:
     * Domain Order -> JPA OrderEntity -> Database
     * Database -> JPA OrderEntity -> Domain Order
     */
    @Override
    public Order save(Order order) {
        // Step 1: Domain -> JPA Entity
        OrderEntity orderEntity = orderDataAccessMapper
            .orderToOrderEntity(order);

        // Step 2: Save to database
        OrderEntity savedEntity = orderJpaRepository.save(orderEntity);

        // Step 3: JPA Entity -> Domain
        return orderDataAccessMapper
            .orderEntityToOrder(savedEntity);
    }

    @Override
    public Optional<Order> findById(OrderId orderId) {
        return orderJpaRepository
            .findById(orderId.getValue())
            .map(orderDataAccessMapper::orderEntityToOrder);
    }
}
```



```
@Override
public Optional<Order> findByTrackingId(TrackingId trackingId) {
    return orderJpaRepository
        .findByTrackingId(trackingId.getValue())
        .map(orderDataAccessMapper::orderEntityToOrder);
}
```

Listing 5: OrderRepositoryImpl - Adapter Pattern

5.2 Domain Entity Implementation

```
/**
 * ORDER ENTITY - Aggregate Root
 *
 * Characteristics:
 * - Aggregate Root: single entry-point to manipulate the Order aggregate
 * - Rich Domain Model: contains data + behavior + business rules
 * - Immutable after creation: getters only; state changes via methods
 */
public class Order extends AggregateRoot<OrderId> {

    // Immutable fields - set once in the constructor
    private final CustomerId customerId;
    private final RestaurantId restaurantId;
    private final StreetAddress deliveryAddress;
    private final Money price;
    private final List<OrderItem> items;

    // Mutable fields - can be changed via business logic
    private TrackingId trackingId;
    private OrderStatus orderStatus;
    private List<String> failureMessages;

    /**
     * BUSINESS LOGIC: Initialize Order
     */
    public void initializeOrder() {
        setId(new OrderId(UUID.randomUUID()));
        trackingId = new TrackingId(UUID.randomUUID());
    }
}
```

```
        orderStatus = OrderStatus.PENDING;
        initializeOrderItems();
    }

    /**
     * BUSINESS LOGIC: Validate Order
     */
    public void validateOrder() {
        validateInitialOrder();
        validateTotalPrice();
        validateItemsPrice();
    }

    /**
     * BUSINESS RULE: Total price must equal the sum of items
     */
    private void validateItemsPrice() {
        // Calculate total from items
        Money orderItemsTotal = items.stream()
            .map(orderItem -> {
                validateItemPrice(orderItem);
                return orderItem.getSubTotal();
            })
            .reduce(Money.ZERO, Money::add);

        // Compare with order price
        if (!price.equals(orderItemsTotal)) {
            throw new OrderDomainException(
                "Total price: " + price.getAmount() +
                " is not equal to Order items total: " +
                orderItemsTotal.getAmount() + "!");
        }
    }

    /**
     * BUSINESS LOGIC: Mark order as PAID
     */
    public void pay() {
        if (orderStatus != OrderStatus.PENDING) {
            throw new OrderDomainException(
```



```
        "Order is not in correct state for pay operation!");  
    }  
    orderStatus = OrderStatus.PAID;  
}  
}
```

Listing 6: Order Entity - Aggregate Root

Domain Entity Best Practices

- **Rich Domain Model:** Entity contains behavior, not just data
- **Business Rules:** All rules live inside the entity
- **Immutability:** Final fields; state changes only via methods
- **Self-validation:** Entity validates its own state
- **No framework dependencies:** Pure Java, no Spring/JPA imports

6 Database Schema

6.1 PostgreSQL Schema Design

```
-- Create schema
CREATE SCHEMA IF NOT EXISTS "order";

-- Create ENUM type for order status
CREATE TYPE order__status AS ENUM (
    'PENDING',
    'PAID',
    'APPROVED',
    'CANCELLING',
    'CANCELLED'
);

-- Orders table (main aggregate)
CREATE TABLE "order".orders (
    id UUID NOT NULL,
    customer_id UUID NOT NULL,
    restaurant_id UUID NOT NULL,
    tracking_id UUID NOT NULL UNIQUE,
    price NUMERIC(10,2) NOT NULL,
    order_status order__status NOT NULL,
    failure_messages VARCHAR,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,
    CONSTRAINT orders_pkey PRIMARY KEY (id)
);

-- Order items table (aggregate members)
CREATE TABLE "order".order_items (
    id BIGINT NOT NULL,
    order_id UUID NOT NULL,
    product_id UUID NOT NULL,
    price NUMERIC(10,2) NOT NULL,
    quantity INTEGER NOT NULL,
    sub_total NUMERIC(10,2) NOT NULL,
    CONSTRAINT order_items_pkey PRIMARY KEY (id, order_id),
    CONSTRAINT fk_order_id FOREIGN KEY (order_id)
        REFERENCES "order".orders(id)
```



```
        ON DELETE CASCADE
    );

-- Indexes for performance
CREATE INDEX idx_orders_tracking_id
    ON "order".orders(tracking_id);

CREATE INDEX idx_orders_customer_id
    ON "order".orders(customer_id);
```

Listing 7: PostgreSQL Database Schema

7 SAGA Pattern

7.1 Distributed Transaction Problem

Trong microservices, một business transaction có thể span qua nhiều services:



Problem: Payment succeeds
but Restaurant approval fails
⇒ Data inconsistency!

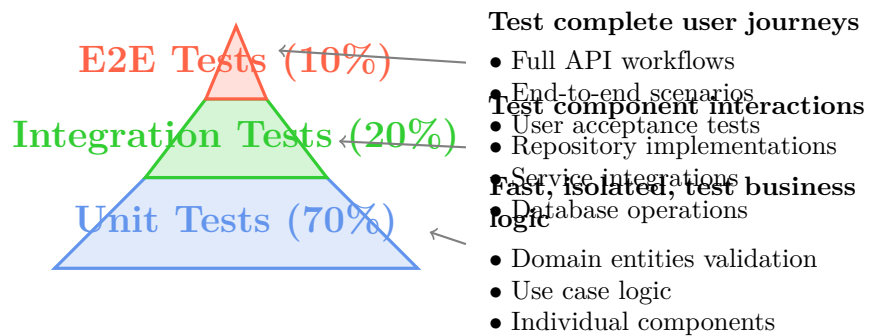
Hình 4: Distributed Transaction Challenge

Distributed Transaction Challenges

- **Two-Phase Commit (2PC):** Slow, blocking, single point of failure
- **Eventual Consistency:** Temporary inconsistency là acceptable
- **Compensating Actions:** Rollback bằng cách gọi "undo" operations

8 Testing Strategy

8.1 Testing Pyramid



Hình 5: Testing Pyramid

Testing Best Practices

- **Unit Tests (70%)**: Fast, isolated, test business logic
- **Integration Tests (20%)**: Test component interactions
- **E2E Tests (10%)**: Test complete user journeys
- **Test Doubles**: Use mocks, stubs, fakes appropriately
- **Test Data Builders**: Create readable test data

8.2 Unit Test Example

```
/**
 * UNIT TEST - Domain Entity
 */
class OrderTest {

    @Test
    @DisplayName("Should validate order successfully with valid data")
    void shouldValidateOrderSuccessfully() {
        // Given
        Order order = createValidOrder();

        // When & Then - no exception thrown
        assertDoesNotThrow(() -> order.validateOrder());
    }
}
```

```
// Verify state
assertThat(order.getOrderStatus()).isEqualTo(OrderStatus.PENDING);
assertThat(order.getId()).isNotNull();
assertThat(order.getTrackingId()).isNotNull();
}

@Test
@DisplayName("Should throw exception when total price is zero")
void shouldThrowExceptionWhenTotalPriceIsZero() {
    // Given
    Order order = Order.builder()
        .customerId(new CustomerId(UUID.randomUUID()))
        .restaurantId(new RestaurantId(UUID.randomUUID()))
        .price(new Money(BigDecimal.ZERO)) // Invalid price
        .items(createValidOrderItems())
        .deliveryAddress(createValidAddress())
        .build();

    // When & Then
    OrderDomainException exception = assertThrows(
        OrderDomainException.class,
        () -> order.validateOrder()
    );

    assertThat(exception.getMessage())
        .contains("Total price must be greater than zero");
}
}
```

Listing 8: Unit Test - Domain Entity

9 Deployment

9.1 Docker Configuration

Listing 9: Multi-stage Dockerfile

```
# Multi-stage build for optimized image
FROM openjdk:17-jdk-alpine AS builder

# Set working directory
WORKDIR /app

# Copy Maven files first (for better caching)
COPY pom.xml .
COPY order-domain/pom.xml order-domain/
COPY order-application-service/pom.xml order-application-service/
COPY order-dataaccess/pom.xml order-dataaccess/
COPY order-messaging/pom.xml order-messaging/
COPY order-container/pom.xml order-container/

# Download dependencies (cached layer)
RUN mvn dependency:go-offline -B

# Copy source code
COPY . .

# Build application
RUN mvn clean package -DskipTests

# Runtime stage
FROM openjdk:17-jre-alpine

# Create non-root user
RUN addgroup -g 1000 appgroup && adduser -u 1000 -G appgroup -s /bin/sh -

# Set working directory
WORKDIR /app

# Copy JAR from builder stage
```



```
COPY --from=builder /app/order-container/target/order-container-1.0.jar a

# Change ownership
RUN chown -R appuser:appgroup /app

# Switch to non-root user
USER appuser

# Expose port
EXPOSE 8181

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD wget --no-verbose --tries=1 --spider http://localhost:8181/actuator

# Start application
ENTRYPOINT [ "java", "-jar", "app.jar" ]
```

9.2 Docker Compose Setup

```
version: '3.8'

services:
  # PostgreSQL Database
  postgres:
    image: postgres:14-alpine
    container_name: order-postgres
    environment:
      POSTGRES_DB: order_db
      POSTGRES_USER: order_user
      POSTGRES_PASSWORD: order_pass
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init-scripts:/docker-entrypoint-initdb.d
    networks:
      - order-network
```



```
# Apache Kafka

zookeeper:
  image: confluentinc/cp-zookeeper:7.4.0
  container_name: order-zookeeper
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181
    ZOOKEEPER_TICK_TIME: 2000
  networks:
    - order-network

kafka:
  image: confluentinc/cp-kafka:7.4.0
  container_name: order-kafka
  depends_on:
    - zookeeper
  ports:
    - "9092:9092"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
  networks:
    - order-network

# Order Service

order-service:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: order-service
  depends_on:
    - postgres
    - kafka
  ports:
    - "8181:8181"
  environment:
    SPRING_PROFILES_ACTIVE: docker
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/order_db
```

```
    SPRING_DATASOURCE_USERNAME: order_user
    SPRING_DATASOURCE_PASSWORD: order_pass
    KAFKA_BOOTSTRAP_SERVERS: kafka:9092

networks:
  - order-network

restart: unless-stopped

volumes:
  postgres_data:

networks:
  order-network:
    driver: bridge
```

Listing 10: Docker Compose Configuration

Quick Start Commands

```
# Build and start all services
docker-compose up --build

# Start in background
docker-compose up -d

# View logs
docker-compose logs -f order-service

# Stop all services
docker-compose down

# Clean up (remove volumes)
docker-compose down -v
```

10 Câu hỏi mở rộng cho sinh viên

10.1 Phần 1: Hiểu rõ Clean Architecture

1. Giải thích Dependency Rule và tại sao nó quan trọng?
2. Sự khác biệt giữa Input Port và Output Port là gì?
3. Tại sao Domain Layer không được phụ thuộc vào Spring Framework?
4. Entity vs Value Object: Khác nhau như thế nào? Cho ví dụ.
5. Aggregate Root là gì? Tại sao Order là Aggregate Root?

10.2 Phần 2: Triển khai thêm tính năng

1. **Update Order:** Implement use case cho phép customer cập nhật đơn hàng (thêm/xóa items) khi order còn ở trạng thái PENDING
2. **Order History:** Tạo API GET /orders?customerId={id} để lấy lịch sử đơn hàng của customer
3. **Order Statistics:** Tạo API GET /orders/statistics để lấy thống kê: tổng đơn, tổng doanh thu, trung bình giá trị đơn
4. **Voucher/Discount:** Thêm tính năng áp dụng mã giảm giá cho đơn hàng
5. **Order Rating:** Sau khi APPROVED, customer có thể rate đơn hàng (1-5 sao)

10.3 Phần 3: Cải thiện hệ thống

1. **Caching:** Implement Redis cache cho Customer và Restaurant data
2. **Retry Mechanism:** Thêm retry logic cho Kafka publishing failures
3. **Rate Limiting:** Implement rate limiting cho API (Ví dụ: max 10 orders/minute per customer)
4. **Async Processing:** Xử lý các tác vụ nặng (validation, notification) bất đồng bộ
5. **Monitoring:** Thêm metrics và alerting cho order creation failures



10.4 Phần 4: Testing

1. Viết unit test cho `Order.validateOrder()` method
2. Viết integration test cho `CreateOrderUseCase`
3. Viết E2E test cho `POST /orders` API
4. Mock các dependencies trong test như thế nào?
5. TestContainers được dùng để làm gì?

10.5 Phần 5: Security

1. Implement JWT authentication cho APIs
2. Authorization: Customer chỉ được xem orders của mình
3. Input validation: Validate tất cả inputs
4. SQL Injection prevention
5. Rate limiting per user



11 Next Steps

11.1 Next Steps cho Sinh viên

Practical Exercises

- **Implement Payment Service:** Tạo service xử lý thanh toán với SAGA compensation
- **Add Restaurant Service:** Service quản lý menu và approve orders
- **Event Sourcing:** Implement event sourcing cho Order aggregate
- **CQRS Pattern:** Tách Command và Query models
- **API Gateway:** Thêm gateway với authentication/authorization
- **Monitoring:** Thêm Prometheus, Grafana, distributed tracing



THUẬT NGỮ VÀ CHỮ VIẾT TẮT

API Application Programming Interface - Giao diện lập trình ứng dụng

CQRS Command Query Responsibility Segregation - Tách biệt trách nhiệm Command và Query

DDD Domain-Driven Design - Thiết kế hướng miền

DIP Dependency Inversion Principle - Nguyên tắc đảo ngược phụ thuộc

DTO Data Transfer Object - Đối tượng truyền tải dữ liệu

E2E End-to-End - Kiểm thử đầu cuối

JPA Java Persistence API - API lưu trữ Java

JWT JSON Web Token - Token web JSON

SAGA Long-running transaction pattern - Mẫu giao dịch dài hạn

TDD Test-Driven Development - Phát triển hướng kiểm thử

UUID Universally Unique Identifier - Định danh duy nhất toàn cầu



TÀI LIỆU THAM KHẢO

1. Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
2. Alistair Cockburn. *Hexagonal Architecture*. 2005.
3. Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
4. Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
5. Spring Framework Documentation. <https://spring.io/projects/spring-boot>
6. Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
7. PostgreSQL Documentation. <https://www.postgresql.org/docs/>